Clean Unit Testing Notes (TDD & Best Practices)

1. Three Laws of TDD

1. You may not write production code until you have written a failing test.

2. You may not write more test than necessary to fail.

3. You may not write more production code than necessary to pass.

Bad Example

```
func Add(a, b int) int {
    return a + b
}
```

Good Example (TDD)

```
func TestAdd(t *testing.T) {
    if Add(2,3) != 5 {
        t.Error("expected 5")
    }
}
```

2. Clean Tests = Readability

Tests must be readable, simple, and expressive.

Bad Test

```
crawler.addPage(root, PathParser.parse("PageOne"))
request.setResource("root")
responder.makeResponse(...)
```

Good Test (DSL)

```
makePages("PageOne", "PageTwo")
submitRequest("root", "type:pages")
assertResponseContains("PageOne")
```

3. Domain-Specific Language (DSL)

Create helper functions to make tests readable.

```
givenUser("Arthur")
whenLoginFails()
thenErrorShouldAppear()
```

4. One Concept per Test

Bad Example

```
func TestEverything(t *testing.T) {
    // multiple unrelated checks
}
```

Good Example

```
func TestAddMonth_ClampTo30(t *testing.T) {}
func TestAddMonth_Return31(t *testing.T) {}
```

5. F.I.R.S.T Principles

Fast: Tests must run quickly.

Independent: No dependency between tests.

Repeatable: Run anywhere.

Self-validating: Automatic pass/fail.

Timely: Write before production code.

6. Mock vs Database

Bad (Using DB in Unit Test)

```
db := connectDB()
user := db.GetUser(1)
```

Good (Using Mock)

```
type MockRepo struct {}
func (m MockRepo) GetUser(id int) User {
    return User{Name: "Arthur"}
}
```